Scenario 3: OOMKilled and Kubernetes Resource Limits

Objective

Use Datadog and Kubernetes evidence to distinguish memory pressure from an unsafe container resource limit.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/shrink-limits.sh support-tickets
```

The script saves the current resources and applies a deliberately unsafe memory request and limit. Wait for restarts and Datadog signals before troubleshooting.

Situation

The support tickets service is unstable. Backend pods restart repeatedly while some requests still succeed.

Symptoms

- Backend pods restart repeatedly.
- Application stability degrades.

What You Know

- Datadog shows backend memory near its configured ceiling.
- Container restart counts are increasing.
- The problem began after a workload resource change.

Start With Datadog

1. Go to **Dashboards > SRE Lab Scenario Signals** and set the time range to **Past 30 Minutes**.
2. In **Memory Usage and Limits**, filter `kube_namespace:support-tickets` and `kube_deployment:support-tickets-backend`. Compare `kubernetes.memory.usage` with `kubernetes.memory.limits` immediately before each restart.
3. In **Container Restarts**, confirm `kubernetes.containers.restarts` increases for the same Deployment.

Record the memory value, configured limit, restart timestamp, pod name, and monitor transition. Confirm the exact `OOMKilled` reason later with `kubectl describe`.

Troubleshooting

1. Identify the restarting pod.

```
kubectl get pods -n support-tickets -l app=support-tickets-backend
```

Expected output: the backend pod is Running again, but its `RESTARTS` count is greater than zero and continues increasing.

1. Inspect the previous container termination.

```
kubectl describe pod <pod-name> -n support-tickets
```

Expected output: under **Last State**, the reason is `OOMKilled` and the exit code is `137`.

1. Compare live memory with the configured limit.

```
kubectl top pod <pod-name> -n support-tickets
kubectl get deployment support-tickets-backend -n support-tickets \
  -o jsonpath='{.spec.template.spec.containers[0].resources}{"\n"}'
```

Expected output: memory usage approaches the deliberately reduced `20Mi` memory limit. The container exceeds that limit and Kubernetes terminates it.

1. Check the previous container logs for supporting evidence.

```
kubectl logs <pod-name> -n support-tickets --previous
```

Expected output: logs stop around the restart time and may not contain a graceful application error. `OOMKilled` in the pod state is the authoritative evidence.

Important Clues

Look for `Reason: OOMKilled`. Compare observed memory consumption with `resources.requests.memory` and `resources.limits.memory`.

Root Cause

Record the relationship between observed memory, the previous container state, and the configured limit. Confirm it with the instructor.

Fix

Restore the known baseline, then use observed utilization to propose a justified long-term request and limit.

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
kubectl rollout status deployment/support-tickets-backend -n support-tickets
```

Do not remove the limit or choose an arbitrary large value.

Validation

Confirm pods remain Ready, restart counts stop increasing, memory stays below the limit, requests succeed, and both Datadog monitors recover.

DevOps Lesson

Resource limits must reflect measured workload behavior. Admission control can accept a value that is operationally unsafe.

STAR Method

Situation

Summarize the unstable workload.

Task

Explain how you separated application, Kubernetes, and infrastructure causes.

Action

Describe the memory, restart, previous-state, and resource comparison.

Result

State how stability and memory headroom were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

An internal backend repeatedly restarted after its memory limit was reduced to 20 MiB. My task was to distinguish an application failure from a Kubernetes resource problem. I correlated Datadog memory and

restart signals with exit code 137 and `OOMKilled` in the pod state, then restored a measured memory limit. The restart loop stopped, pods remained Ready, and future resource values were based on observed usage.